*INSTITUTO POLITÉCNICO NACIONAL*
***Unidad Profesional Interdisciplinaria de Ingeniería y Ciencias Sociales y Administrativas***

# *Fundamentos de Ingeniería de Software*

***Prof. Paula León A.***

## *Actividad 14. Gestión de la calidad*

***Objetivo***: Identificar la importancia de los estándares en el proceso de gestión de calidad y aprender cómo se usan los estándares en el aseguramiento de calidad en nuestro país.

**Fecha:** _____

**Nombre(s):** _____ **No. Lista** _____ **Grupo:** _____

**(Apellido paterno. Apellido materno. Nombre(s)**

- **ISO.** Organización Internacional de Estandarización.
- **IEC**. Comisión Electrotécnica Internacional. Es una organización de normalización en los campos eléctrico, electrónico y de tecnologías relacionadas.
- **ANSI.** El Instituto Nacional Estadounidense de Estándares, (por sus siglas en inglés: American National Standards Institute), es una organización sin fines de lucro que supervisa el desarrollo de estándares para productos, servicios, procesos y sistemas en los Estados Unidos.
- **US.**DOD. USDOD o DOD. Departamento de Defensa de los Estados Unidos (del inglés: *United States Department of Defense*) es un departamento ejecutivo federal del gobierno de Estados Unidos encargado de coordinar y supervisar todas las agencias y funciones del gobierno directamente relacionadas con la seguridad nacional.
- **BSI.** British Standards Institution, BSI o BSI Group, es una multinacional con sede en Londres, cuyos objetivos principales se incluyen la certificación, auditoría y formación en las normas para estandarizar procesos. Forma parte de la Organización Internacional de Normalización (ISO) y proveedor de estas, entre las que destaca las ISO 9000, ISO 14000, ISO 27000.
- **Serie ISO/IEC 27000** de normas son estándares de seguridad publicados por la Organización Internacional para la Estandarización (ISO) y la Comisión Electrotécnica Internacional (IEC). La serie contiene las mejores prácticas recomendadas en Seguridad de la información para desarrollar, implementar y mantener especificaciones para los Sistemas de Gestión de la Seguridad de la Información (SGSI).
- **OTAN.** Organización del Tratado del Atlántico Norte es una organización internacional de carácter político y militar cuyo objetivo es garantizar la libertad y la seguridad de sus miembros a través de medios políticos y militares.
- **IEEE.** Instituto de Ingenieros Eléctricos y Electrónicos (Institute of Electrical and Electronics Engineers). Es una asociación mundial de ingenieros dedicada a la normalización y el desarrollo en áreas técnicas.
- **NYCE:** Organismo Nacional de Estandarización para las industrias de Electrónica, Telecomunicaciones, Tecnologías de la Información, Química y Plásticos. Presta servicios de certificación, verificación y capacitación en Normas Oficiales Mexicanas (NOM), Normas Mexicanas (NMX) y estándares internacionales ISO/IEC.



**Norma ISO/IEC 9126** permite especificar y evaluar la **calidad** del **software** desde diferentes criterios asociados con adquisición, requerimientos, desarrollo, uso, evaluación, soporte, mantenimiento, aseguramiento de la **calidad** y auditoría de **software.**

**NMX-I-9126-2-NYCE-2011**
Tecnología de la información-ingeniería de software-calidad de producto-parte 1: modelo de calidad (CANCELA A LA NMX-I-055/01-NYCE-2006).
Esta Norma coincide totalmente con la Norma Internacional ISO/IEC 9126-1:2001, *Edition 1, Software engineering - Product quality - Part 1: Quality model*.
A través de la norma **NMX-I-9126-2-NYCE-2011-Tecnología de la información-***Ingeniería de Software – Calidad De Producto- Métricas de externas*, es posible comprobar la característica de **Funcionalidad** de un producto software que involucra características como:

- **Adecuación:** La capacidad de un producto software para proporcionar un conjunto adecuado de funciones que desempeñan tareas y objetivos específicos para los usuarios para los que fue diseñado el software.
- **Exactitud:** La capacidad un producto de software para proporcionar los resultados o efectos correctos o acordados en la especificación de requisito, con el grado necesario de exactitud.
- **Interoperabilidad:** La capacidad un producto software para interactuar y comunicarse con uno o más productos de software o a otros sistemas.
- **Seguridad de acceso:** La capacidad de un producto software para proteger información y datos de manera que las personas o sistemas no autorizados no puedan leerlos o modificarlos, al tiempo que no se deniega el acceso a las personas o sistemas autorizados.
- **Cumplimiento funcional:** La capacidad de un producto software para adherirse a normas, convenciones o legislación y prescripciones similares relacionadas con la funcionalidad.

NYCE
Seguridad y Confianza
CLICK

## 1. Introducción

- La **gestión de calidad del software** se ocupa de garantizar que el software tenga un número menor de defectos y que alcance los estándares requeridos de mantenibilidad, fiabilidad, portabilidad, etc. Incluye definir estándares para procesos y productos, y establecer procesos para comprobar que se siguieron dichos estándares.
- Los estándares de software son importantes para el aseguramiento de la calidad, pues representan una identificación de las "mejores prácticas". Al desarrollar el software, los estándares proporcionan un cimiento sólido para diseñar software de buena calidad.
- Es necesario documentar un conjunto de procedimientos de aseguramiento de la calidad en un manual de calidad organizacional. Esto puede basarse en el modelo genérico para un manual de calidad sugerido en el estándar ISO 9001.
- Las revisiones de los entregables del proceso de software incluyen a un equipo de personas que verifican que se siguieron los estándares de calidad. Las revisiones son la técnica usada más ampliamente para valorar la calidad.
- En una inspección de programa o revisión de pares, un reducido equipo comprueba sistemáticamente el código. Ellos leen el código a detalle y buscan posibles errores y omisiones. Entonces los problemas detectados se discuten en una reunión de revisión del código.
- La medición del software puede usarse para recopilar datos cuantitativos tanto del software como del proceso de software. Se usan los valores de las métricas de software recopilados para hacer inferencias referentes a la calidad del producto y el proceso.
- Las métricas de calidad del producto son útiles para resaltar los componentes anómalos que pudieran tener problemas de calidad. Dichos componentes deben entonces analizarse con más detalle.

## 2. Gestión de la calidad y desarrollo de software

Cuando se habla de la calidad del software profesional, se debe considerar que el software lo usan y cambian personas, además de sus desarrolladores. En consecuencia, la calidad no tiene que ver sólo con lo que hace el software. Debe incluir el comportamiento del software mientras se ejecuta, y la estructura y organización de los programas del sistema y la documentación asociada. Esto se refleja en los llamados calidad o atributos no funcionales del software.

Ejemplos de dichos atributos son el tiempo de respuesta del software ante la duda de un usuario y la comprensibilidad del código del programa.

El conjunto específico de atributos que se espera de un sistema de software depende evidentemente de su aplicación. Así, un sistema bancario debe ser seguro, un juego interactivo debe tener capacidad de respuesta, un sistema de conmutación telefónica debe ser confiable, etc. Esto puede generalizarse en el conjunto de atributos que se muestra en la siguiente figura los cuales consideran las características esenciales de un sistema de software profesional.

| Características del producto | Descripción |
|---|---|
| Mantenimiento | El software debe escribirse de tal forma que pueda evolucionar para satisfacer las necesidades cambiantes de los clientes. Éste es un atributo crítico porque el cambio del software es un requerimiento inevitable de un entorno empresarial variable. |
| Confiabilidad y seguridad | La confiabilidad del software incluye un rango de características que abarcan fiabilidad, seguridad y protección. El software confiable no tiene que causar daño físico ni económico, en caso de falla del sistema. Los usuarios malintencionados no deben tener posibilidad de acceder al sistema o dañarlo. |
| Eficiencia | El software no tiene que desperdiciar los recursos del sistema, como la memoria y los ciclos del procesador. Por lo tanto, la eficiencia incluye capacidad de respuesta, tiempo de procesamiento, utilización de memoria, etcétera. |
| Aceptabilidad | El software debe ser aceptable al tipo de usuarios para quienes se diseña. Esto significa que necesita ser comprensible, utilizable y compatible con otros sistemas que ellos usan. |

***Características esenciales de un sistema de software profesional***

La ingeniería busca obtener resultados de la calidad requerida dentro de la fecha y del presupuesto. Sin embargo, las personas que diseñan programas para sí mismas podrían pasar tanto tiempo como deseen en el desarrollo del programa. En general, los ingenieros de software adoptan en su trabajo un enfoque sistemático y organizado, pues usualmente ésta es la forma más efectiva de producir software de alta calidad. La ingeniería busca seleccionar el método más adecuado para un conjunto de circunstancias y de esta manera, un acercamiento al desarrollo más creativo y menos formal sería efectivo en ciertas situaciones.

Los problemas de calidad del software se descubrieron inicialmente en la década de **1960** con el desarrollo de los primeros grandes sistemas de software y han prevalecido en la ingeniería de software a partir de esa década. El software entregado era lento y poco fiable, difícil de mantener y de reutilizar. El descontento con esta situación condujo a la adopción de técnicas formales de gestión de calidad del software, desarrolladas a partir de métodos usados en la industria manufacturera. Estas técnicas de gestión de calidad, en conjunto con nuevas tecnologías y mejores pruebas de software, llevaron a progresos significativos en el nivel general de calidad del software.

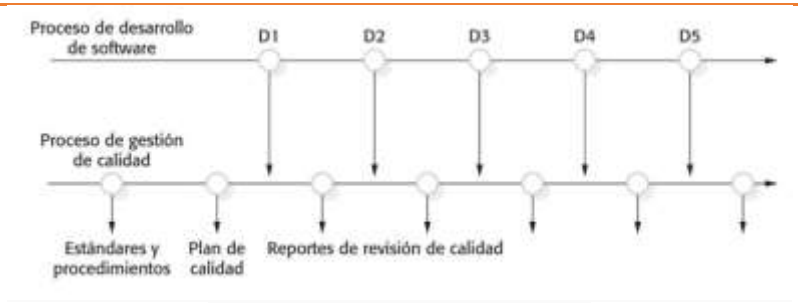
La gestión de calidad del software para los sistemas de software tiene **tres intereses fundamentales**:

1. A nivel de **organización,** la gestión de calidad se ocupa de establecer un marco de proceso y estándares de organización que conducirán a software de mejor calidad. Esto supone que el equipo de gestión de calidad debe tener la responsabilidad de definir los procesos de desarrollo del software a usar, los estándares que deben aplicarse al software y la documentación relacionada, incluyendo los requerimientos, el diseño y el código del sistema.
2. A nivel del **proyecto,** la gestión de calidad implica la aplicación de procesos específicos de calidad y la verificación de que continúen dichos procesos planeados; además, se ocupa de garantizar que los resultados del proyecto estén en conformidad con los estándares aplicables a dicho proyecto.
3. A nivel del **proyecto**, la gestión de calidad se ocupa también de establecer un plan de calidad para un proyecto. El plan de calidad debe establecer metas de calidad para el proyecto y definir cuáles procesos y estándares se usarán.

Los términos aseguramiento de calidad y control de calidad se utilizan ampliamente en la industria manufacturera. El aseguramiento de calidad (**QA**, por las siglas de Quality Assurance) es la definición de procesos y estándares que deben conducir a la obtención de productos de alta calidad y, en el proceso de fabricación, a la introducción de procesos de calidad. El control de calidad es la aplicación de dichos procesos de calidad para eliminar aquellos productos que no cuentan con el nivel requerido de calidad.

**En la industria de software, diversas compañías y sectores industriales interpretan de maneras diferentes el aseguramiento de calidad y el control de calidad**. En ocasiones, el aseguramiento de calidad representa simplemente la definición de procedimientos, procesos y estándares cuyo objetivo es asegurar el logro de calidad del software. En otros casos, el aseguramiento de calidad incluye también todas las actividades de gestión de configuración, verificación y validación aplicadas después de que un equipo de desarrollo entrega un producto. Se usa el término **aseguramiento de calidad** para incluir verificación y validación, y los procesos de que la comprobación de procedimientos de calidad se aplicó de manera adecuada. No confundir con **"control de calidad"**, puesto que esta expresión no se usa mucho en la industria del software.

En la mayoría de las compañías, el equipo QA es el responsable de administrar el proceso de pruebas de **liberación**, esto significa que **se aplican las pruebas del software antes de que éste se libere a los clientes**. El equipo es responsable de comprobar que las pruebas del sistema cubran los requerimientos y de mantener los registros adecuados del proceso de pruebas.




***Gestión de calidad y desarrollo de software***

La gestión de calidad proporciona una comprobación independiente sobre el proceso de desarrollo de software. El proceso de gestión de calidad verifica los entregables del proyecto para garantizar que sean consistentes con los estándares y las metas de la organización. El equipo QA debe ser independiente del equipo de desarrollo para que pueda tener una perspectiva objetiva del software. Esto les permite reportar la calidad del software sin estar influidos por los conflictos de desarrollo del software.

De preferencia, el equipo de gestión de calidad no debe asociarse con algún grupo de desarrollo particular; sin embargo, tiene la responsabilidad ante toda la organización por la administración de la calidad. El equipo debe ser independiente y reportarse ante la administración ubicada sobre el nivel del administrador del proyecto. La razón es que los administradores de proyecto tienen que mantener el presupuesto y el calendario del proyecto. Si surgen problemas, pueden estar tentados a comprometer la calidad del producto para cumplir con el calendario. Un equipo de gestión de calidad independiente garantiza que las metas de calidad de la organización no estén comprometidas a corto plazo por consideraciones de presupuesto y calendario. Sin embargo, en compañías más pequeñas, esto es prácticamente imposible. La gestión de calidad y el desarrollo de software están vinculados con las personas que tienen responsabilidades tanto de desarrollo como de calidad.

La planeación de calidad es el proceso de desarrollar un plan de calidad para un proyecto. El plan de calidad debe establecer las cualidades deseadas de software y describir cómo se valorarán. Por lo tanto, define lo que realmente significa software de “alta calidad” para un sistema particular. Sin esta definición, los ingenieros pueden hacer diferentes suposiciones, algunas veces conflictivas, sobre cuáles atributos del producto reflejan las características de calidad más importantes.

La planeación de calidad formalizada es parte integral de los procesos de desarrollo basados en un plan. No obstante, **los métodos ágiles adoptan un enfoque menos formal** para la gestión de calidad.

Humphrey (1989), en su clásico libro referente a la gestión del software, sugiere un bosquejo de **estructura para un plan de calidad**. Éste incluye:

1. ***Introducción del producto***. Una descripción del producto, la pretensión de su mercado y las expectativas de calidad para el producto.
2. ***Planes del producto***. Indican las fechas de entrega críticas y las responsabilidades para el producto, junto con planes para distribución y servicio al producto.
3. ***Descripciones de proceso***. Describen los procesos y estándares de desarrollo y servicio que deben usarse para diseño y gestión del producto.
4. ***Metas de calidad*.** Las metas y los planes de calidad para el producto, incluyendo una identificación y justificación de los atributos esenciales de calidad del producto.
5. ***Riesgos y gestión del riesgo***. Los riesgos clave que pueden afectar la calidad del producto y las acciones a tomar para enfrentar dichos riesgos.

Los planes de calidad, que se desarrollan como parte del proceso de planeación general del proyecto, difieren en detalle dependiendo del tamaño y tipo de sistema que se desarrolló. Sin embargo, cuando escriba planes de calidad, debe tratar de mantenerlos tan breves como sea posible. Si el documento es demasiado amplio, las personas no lo leerán y, en consecuencia, se anulará el propósito de generar un plan de calidad.

Algunas personas consideran que la calidad del software puede lograrse mediante procesos establecidos basados en estándares de organización y procedimientos de calidad asociados que verifican el seguimiento de dichos estándares mediante el equipo de desarrollo de software. Su argumento es que los estándares se dirigen a la **buena práctica de ingeniería de software** y que seguir esta buena práctica conducirá a productos de alta calidad. No obstante, en la práctica, se considera que en la gestión de calidad hay mucho más que estándares y burocracia asociados para asegurar que éstos se sigan.

Aunque los estándares y procesos son importantes, los administradores de calidad deben enfocarse también a desarrollar una **"cultura de calidad"** en la que todo responsable del desarrollo del software se comprometa a lograr un alto nivel de calidad del producto. Deben exhortar a los equipos a asumir la responsabilidad de la calidad de su trabajo y desarrollar nuevos enfoques para el mejoramiento de la calidad.

A pesar de que los estándares y procedimientos son la base de la gestión de calidad, los buenos administradores de calidad reconocen que existen aspectos intangibles a la calidad del software (elegancia, legibilidad, etc.) que no pueden expresarse en estándares. Deben apoyar a la gente interesada en aspectos intangibles de calidad e impulsar el comportamiento profesional en todos los miembros del equipo.

La gestión de la calidad formalizada es particularmente importante para los equipos que diseñan grandes sistemas de larga duración, los cuales tardan varios años en desarrollarse. La **documentación de la calidad** es un registro de lo que cada subgrupo realizó en el proyecto. Ayuda a las personas a comprobar que no se olvidaron tareas importantes o que un grupo no hizo suposiciones incorrectas acerca de lo que hicieron otros grupos. La documentación de la calidad también es un medio de comunicación durante la vida del sistema. Permite a los grupos responsables de la evolución del sistema encontrar las pruebas y comprobaciones que el equipo de desarrollo debe implementar.

Para sistemas más pequeños, la gestión de calidad es aún importante, aunque puede adoptarse un enfoque más informal. No se necesita tanto papeleo, puesto que un equipo de desarrollo pequeño puede comunicarse de manera informal. El aspecto de calidad clave para el desarrollo de sistemas pequeños es establecer una cultura de calidad y asegurarse de que todos los miembros del equipo tienen un enfoque positivo sobre la calidad del software.

## 3. Calidad del software

La industria manufacturera estableció los fundamentos de la gestión de calidad para mejorar ésta en los productos que se fabricaban. Como parte de ello se desarrolló una definición de calidad, que se basa en la conformidad con una especificación de producto detallada (Crosby, 1979) y la noción de tolerancia. La suposición subyacente era que los productos podían especificarse por completo y establecerse procedimientos que comprobaran si un producto manufacturado cumplía o no con su especificación. Desde luego, los productos nunca cumplirán exactamente una especificación, pues se permite cierta tolerancia. Si el producto era "casi bueno", se clasificaba como aceptable.

**La calidad del software no es directamente comparable con la calidad en la fabricación**. La idea de tolerancia no es aplicable a los sistemas digitales y **es prácticamente imposible llegar a una conclusión objetiva sobre si un sistema de software cumple o no su especificación, por las siguientes razones:**

1. En cuanto a la **ingeniería de requerimientos**, es difícil escribir especificaciones de software completas y sin ambigüedades. Los desarrolladores y clientes de software pueden interpretar los requerimientos de diferentes formas y tal vez sea imposible llegar a acuerdos acerca de si el software se desarrolló conforme a su especificación.
2. Por lo general, las **especificaciones integran requerimientos de varias clases de participantes**. Dichos requerimientos son un compromiso ineludible y tal vez no incluyan los requerimientos de todos los grupos de participantes. Por lo tanto, las partes interesadas excluidas quizá perciban el sistema como uno de mala calidad, a pesar de que implementa los requerimientos acordados.
3. Es **imposible medir de manera directa o exacta ciertas características de calidad** (por ejemplo, adaptabilidad, flexibilidad, reusabilidad) y, por lo tanto,, no pueden especificarse plenamente sin ambigüedades.
4. Debido a estos problemas**, la valoración de calidad del software es un proceso subjetivo** en que el equipo de gestión de calidad tiene que usar su juicio para decidir si se logró un nivel aceptable de calidad. El equipo de gestión de calidad debe considerar si el software se ajusta o no a su propósito pretendido. Esto implica responder preguntas sobre las características del sistema. Por ejemplo:

**a.** ¿En el proceso de desarrollo se siguieron los estándares de programación y documentación?
**b.** ¿El software se verificó de manera adecuada?
**c.** ¿El software es suficientemente confiable para utilizarse?
**d.** ¿El rendimiento del software es aceptable para uso normal?
**e.** ¿El software es utilizable?
**f.** ¿El software está bien estructurado y es comprensible?

Existe la suposición general en la gestión de calidad del software de que el sistema se pondrá a prueba en contra de sus requerimientos. La decisión acerca de si entregar o no la funcionalidad requerida debe basarse en los resultados de dichas pruebas. Por lo tanto, el equipo QA debe revisar las pruebas que se desarrollaron y examinar los registros de pruebas para verificar que éstas se hayan realizado de manera apropiada. En algunas organizaciones el equipo de gestión de calidad es responsable de las pruebas del sistema, pero, en ocasiones, un grupo de pruebas de sistema separado es responsable de esto.

La **calidad subjetiva** de un sistema de software se basa principalmente en sus **características no funcionales**.

Esto refleja la experiencia práctica del usuario: Si la funcionalidad del software no es lo que se esperaba, entonces los usuarios con frecuencia sólo le darán la vuelta a este asunto y encontrarán otras formas de hacer lo que quieren. Sin embargo, si el software no es fiable o resulta muy lento, entonces es prácticamente imposible que los usuarios logren sus metas.

Por consiguiente, la calidad del software **no sólo se trata de si la funcionalidad** de éste se implementó correctamente, **sino también depende de los atributos no funcionales del sistema**. **Boehm y sus colaboradores** (1978) indican que existen **15 importantes atributos de calidad de software**. Se listan a continuación:

| | | |
|---|---|---|
| Protección | Comprensibilidad | Portabilidad |
| Seguridad | Comprobabilidad | Usabilidad |
| Fiabilidad | Adaptabilidad | Reusabilidad |
| Flexibilidad | Modularidad | Eficiencia |
| Robustez | Complejidad | Facilidad para que el usuario aprenda a utilizarlo |

***Atributos de calidad del software***

Dichos atributos se relacionan con la confiabilidad, usabilidad, eficiencia y mantenibilidad del software. Se considera que los **atributos de confiabilidad** son los atributos de calidad más importantes de un sistema. Sin embargo, también es significativo el rendimiento del software. Los usuarios rechazarán el software que sea demasiado lento. No es posible que algún sistema se optimice para todos esos atributos; por ejemplo, mejorar la robustez puede conducir a pérdida de rendimiento.

En consecuencia, el plan de calidad debe definir los atributos de calidad más importantes para el software que se desarrollará. Tal vez la eficiencia sea crítica y tengan que sacrificarse otros factores para que se logre esto. Si lo anterior se estableció en el plan de calidad, los ingenieros que trabajan en el desarrollo pueden cooperar para lograrlo. El plan debe incluir también una definición del proceso de valoración de la calidad. Ésta debe ser una forma acordada de valorar si cierto grado de calidad, como la mantenibilidad o robustez, está presente en el producto.

Una suposición que subyace en la gestión de la calidad del software es que la calidad del software se relaciona directamente con la calidad del proceso de desarrollo del software. Esto proviene de nuevo de los sistemas fabriles, donde la calidad del producto está estrechamente relacionada con el proceso de producción. Un proceso de fabricación incluye configurar, establecer y operar las máquinas implicadas en el proceso. Una vez que las máquinas operan correctamente, se sigue de manera natural la calidad del producto. Entonces se mide la calidad del producto y el proceso se modifica hasta que se logra el nivel de calidad necesario. La figura siguiente ilustra este enfoque basado en el proceso para obtener la calidad del producto.




***Calidad basada en el proceso***

En la **industria existe un vínculo entre el proceso y la calidad del producto**, ya que el proceso es relativamente sencillo de estandarizar y monitorizar. Una vez calibrados los sistemas de fabricación, pueden operar una y otra vez para generar productos de alta calidad; sin embargo, **el software no se manufactura**, **se diseña**. Por lo tanto, en **el desarrollo del software es más compleja la relación entre calidad de proceso y calidad del producto**. El diseño del software es un proceso creativo más que mecánico, pues es significativa la influencia de las habilidades y la experiencia individuales. Factores externos, como la novedad de una aplicación o la premura por el lanzamiento comercial de un producto, también afectan la calidad de éste sin importar el proceso usado.

El proceso de desarrollo utilizado tiene una influencia importante sobre la calidad del software, y que los buenos procesos tienen más probabilidad de conducir a software de buena calidad. La gestión de la calidad y el mejoramiento del proceso pueden conducir a menores defectos en el software a desarrollar. Es difícil valorar los atributos de calidad del software, como la complejidad, usabilidad, adaptabilidad , sin usar el software durante un largo periodo. En consecuencia, es difícil decir cómo las características del proceso influyen en dichos atributos. Debido al papel del diseño y la creatividad en el proceso de software, la estandarización del proceso en ocasiones puede exterminar la creatividad, lo cual, lejos de elevar la calidad, conducirá a un software de calidad inferior.

## 4. Estándares de software

Los **estándares de software** tienen una función importante en la gestión de calidad del software. Un aspecto importante del aseguramiento de calidad es la definición o selección de estándares que deben aplicarse al proceso de desarrollo de software o al producto de software. Como parte de este proceso QA, también pueden elegirse herramientas y métodos para apoyar el uso de dichos estándares. Una vez seleccionados estos para su uso, deben definirse procesos específicos de proyecto para monitorizar el uso de los estándares y comprobar que éstos se siguieron.

Los **estándares de software son importantes** por tres razones:

1. Reflejan la sabiduría que es de valor para la organización. Se basan en conocimiento sobre la mejor o más adecuada práctica para la compañía. Con frecuencia, este conocimiento se adquiere sólo después de gran cantidad de ensayo y error. Configurarla dentro de un estándar ayuda a la compañía a reutilizar esta experiencia y a evitar errores del pasado.
2. Proporcionan un marco para definir, en un escenario particular, lo que significa el término **“calidad”**. La calidad del software es subjetiva, y al usar estándares **se establece una base para decidir si se logó un nivel de calidad requerido**. Esto depende del establecimiento de estándares que reflejen las expectativas del usuario para la confiabilidad, la usabilidad y el rendimiento del software.

3. Los estándares auxilian la continuidad cuando una persona retoma el trabajo iniciado por alguien más. Aseguran que todos los ingenieros dentro de una organización adopten las mismas prácticas y se reduce el esfuerzo de aprendizaje requerido al iniciarse un nuevo trabajo.

Existen **dos tipos de estándares de ingeniería de software** relacionados que pueden definirse y usarse en la gestión de calidad del software:

1. ***Estándares del producto*.** Se aplican al producto de software a desarrollar. Incluyen estándares de documentos (como la estructura de los documentos de requerimientos), estándares de documentación (como el encabezado de un comentario estándar para una definición de clase de objeto) y estándares de codificación, los cuales definen cómo debe usarse un lenguaje de programación.
2. ***Estándares de proceso*.** Establecen los procesos que deben seguirse durante el desarrollo del software. Deben especificar cómo es una buena práctica de desarrollo. Los estándares de proceso pueden incluir definiciones de especificación, procesos de diseño y validación, herramientas de soporte de proceso y una descripción de los documentos que deben escribirse durante dichos procesos.

| Estándares de producto | Estándares de proceso |
|---|---|
| Formato de revisión de diseño | Realizar revisión de diseño |
| Estructura de documento de requerimientos | Enviar nuevo código para construcción de sistema |
| Formato de encabezado por método | Proceso de liberación de versión |
| Estilo de programación Java | Proceso de aprobación del plan del proyecto |
| Formato de plan de proyecto | Proceso de control de cambio |
| Formato de solicitud de cambio | Proceso de registro de prueba |

***Estándares de producto y proceso***

Los estándares deben entregar valor, en la forma de calidad aumentada del producto. No hay razón para definir estándares que sean costosos en términos de tiempo y esfuerzo, pues aplicarlos sólo conduce a mejoras secundarias en la calidad. Los estándares de producto deben diseñarse de forma que puedan aplicarse y comprobarse de manera efectiva en cuanto a costos, y los estándares de proceso deben incluir la definición de procesos que comprueben que se siguieron dichos estándares.

El desarrollo de **estándares internacionales de ingeniería de software, por lo general, es un proceso prolongado** en el que se reúnen los interesados en el estándar, elaboran borradores para comentar y, **finalmente acuerdan el estándar.**

Organismos nacionales e internacionales, como U.S. DoD, ANSI, BSI, OTAN y el IEEE, apoyan la determinación de estándares. Se trata de estándares generales que pueden aplicarse a través de varios proyectos. Entidades tales como la OTAN y otras organizaciones de defensa pueden requerir que sus propios estándares se usen en el desarrollo de contratos que suscriben con compañías de software.

**Se han desarrollado estándares nacionales e internacionales que incluyen la terminología de ingeniería de software, lenguajes de programación como Java y C++, anotaciones como los símbolos de diagramación, procedimientos para derivar y escribir requerimientos de software, procedimientos de aseguramiento de calidad, y procesos de verificación y validación de software (IEEE, 2003). Estándares más especializados, como IEC 61508 (IEC, 1998), se desarrollaron para sistemas críticos de protección y seguridad.**

Los equipos de gestión de calidad que elaboran estándares para alguna compañía, por lo general deben basar los estándares de dicha compañía en estándares nacionales e internacionales. Al usar estándares internacionales como punto de partida, el equipo de aseguramiento de calidad debe redactar un manual de estándares, el cual debe definir los estándares que necesita su organización.

En ocasiones, los ingenieros de software consideran los estándares como demasiado prescriptivos y realmente poco relevantes para la actividad técnica del desarrollo de software. Esto es probable sobre todo cuando los estándares de proyecto requieren documentación y registro del trabajo tediosos. Aunque en general están de acuerdo con la necesidad de los estándares, los ingenieros encuentran a menudo razones para señalar que los estándares no necesariamente son adecuados para su proyecto particular. Para minimizar el descontento y alentar la participación en los estándares, los administradores de calidad que establezcan los estándares deben dar los siguientes pasos:

1. Involucrar a los ingenieros de software en la selección de estándares de producto Si los desarrolladores comprenden por qué se seleccionaron los estándares, tienen más probabilidad de comprometerse con éstos. De preferencia, los documentos de estándares no deben establecer sólo el estándar a seguir, sino también deben incluir comentarios que expliquen por qué se tomaron las decisiones de estandarización.
2. Revisar y modificar regularmente los estándares para reflejar las tecnologías cambiantes. Los estándares son costosos de desarrollar y tienden a guardarse como reliquias en un manual de estándares de una compañía. Debido a los costos y la discusión requeridos, muchas veces hay reticencia para cambiarlos. Aunque un manual de estándares es esencial, debe evolucionar para reflejar las circunstancias y la tecnología cambiantes.

3. Ofrecer herramientas de software para dar soporte a los estándares. Los desarrolladores encuentran con frecuencia que los estándares son una pesadilla cuando la adhesión a ellos incluye un tedioso trabajo manual que podría hacerse mediante una herramienta de software. Si está disponible el soporte para herramientas, se requiere muy poco esfuerzo para seguir los estándares de desarrollo de software. Por ejemplo, los estándares de documento pueden implementarse mediante estilos de procesador de texto.

   Diferentes tipos de software necesitan distintos procesos de desarrollo, puesto que los estándares deben ser adaptables. No hay razón para prescribir una forma particular de trabajar si es inadecuada para un proyecto o equipo de proyecto. Cada administrador de proyecto debe tener la autoridad de modificar los estándares de proceso de acuerdo con las circunstancias individuales. Sin embargo, cuando se hacen cambios, es importante garantizar que dichos cambios no conduzcan a una pérdida de calidad del producto. Esto afectará la relación de una empresa con sus clientes y conducirá probablemente a un aumento en los costos del proyecto.

El **administrador del proyecto y el administrador de calidad pueden evitar problemas en los estándares** al planear cuidadosamente la calidad oportuna en el proyecto. Deben decidir cuál de los estándares de la organización debe usarse sin cambio, cuáles deben modificarse y cuáles ignorarse. Es posible que deban crearse nuevos estándares en res- puesta a requerimientos del cliente o del proyecto. Por ejemplo, tal vez se requieran estándares para especificaciones formales si no se han usado en proyectos anteriores.

## a. Marco de estándares ISO 9001

Como los sistemas de información son vulnerables a accesos no autorizados, con la consiguiente alteración del contenido de la información, los datos deben permanecer íntegros, confiables, auténticos y poder ser utilizados cuando sea necesario.

Existen los sistemas de administración de seguridad de información (**ISMS**, por sus siglas en inglés) los cuales consisten de políticas, procedimientos, recursos humanos y materiales. Este tipo de sistemas deben mantener los datos de manera confidencial, íntegra y disponible en cualquier momento. Al hacer referencia a datos confidenciales, significa que sólo están disponibles para personas autorizadas; íntegros, significa que los datos no pueden ser alterados, excepto por las personas autorizadas; y disponibilidad de datos significa que estarán disponibles en cualquier momento pero sólo para personas autorizadas.

La **Organización Internacional de Estandarización** (**ISO**, por sus siglas en inglés) ha elaborado y emitido normas ISO/IEC 27000 que complementan el uso e implementación de un ISMS. Los requerimientos de la norma 27000 se pueden aplicar a cualquier tipo de organización, sin importar su tamaño, sector al que pertenece u objetivos de la organización, y declaran cómo establecer, administrar, documentar y mejorar de manera continua un ISMS, con un enfoque basado en la

administración de riesgo, pues sólo controlando el riesgo que corren los datos es posible garantizar su confidencialidad. Se puede auditar un ISMS contra el estándar de la norma ISO/IEC 27001 y obtener una certificación.

La norma define 130 controles de riesgo que cubren prácticamente todas las áreas de la empresa donde existe una amenaza potencial a la seguridad de los datos. Una vez identificados y analizados los riesgos potenciales, se elaboran las políticas, los procedimientos y la documentación de todos los procesos involucrados en la prevención de los riesgos, lo cual incluye políticas de seguridad TI utilizada, controles de acceso a la TI y a los datos, de acuerdo con los requerimientos legales y estándares de la norma. Los 130 controles no son exhaustivos y dependiendo de la situación y la empresa se pueden eliminar algunos y agregar otros más allá de los 130 que declara la norma y se adecuen a las particularidades de la organización.

Por lo tanto, para que la información tenga **confidencialidad**, los procesos del manejo de la información deben ser óptimos, el personal que maneja la información debe estar capacitado, se debe tener tanto claves de acceso al sistema como encriptamiento de la información y de preferencia debe obtenerse la certificación en la Norma ISO/IEC 27000 y 27000.

Existe un conjunto internacional de estándares que pueden utilizarse en el desarrollo de los sistemas de administración de calidad en todas las industrias, llamado ISO 9000. Los estándares ISO 9000 pueden aplicarse a varias organizaciones, desde las industrias manufactureras hasta las de servicios. ISO 9001, el más general de dichos estándares, se aplica a organizaciones que diseñan, desarrollan y mantienen productos, incluido software. El estándar ISO 9001 se desarrolló originalmente en 1987, y su revisión más reciente fue en 2008.

El estándar **ISO 9001 no es en sí mismo un estándar** para el desarrollo de software, **sino un marco para elaborar estándares de software**. Establece principios de calidad total, describe en general el proceso de calidad, y explica los estándares y procedimientos organizacionales que deben determinarse. Éstos tienen que documentarse en un manual de calidad de la organización.

La revisión principal del estándar ISO 9001 reorientó en 2000 el estándar hacia nueve procesos centrales. Se ilustran a continuación:

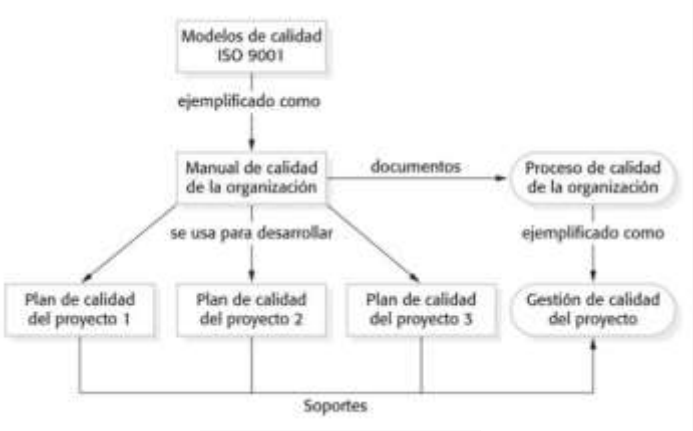



**Procesos centrales ISO 9001**

Si una organización quiere estar conforme con el estándar ISO 9001, debe documentar cómo se relacionan sus procesos con dichos procesos centrales. También deberá definir y mantener registros que demuestren que se siguieron los procesos organizacionales establecidos. El manual de calidad de la compañía tiene que describir los procesos relevantes y los datos de proceso que deben recopilarse y conservarse.

El estándar ISO 9001 no define ni prescribe los procesos de calidad específicos que deben usarse en una compañía. Para estar de conformidad con ISO 9001, una compañía debe especificar los tipos de proceso y tener procedimientos que demuestren que se siguen sus procesos de calidad. Esto permite flexibilidad a través de sectores industriales y diversos tamaños de compañías. Pueden definirse estándares de calidad que sean adecuados para el tipo de software a desarrollar.
Las compañías pequeñas pueden tener procesos no burocráticos y estar en conformidad con ISO 9001. Sin embargo, esta flexibilidad significa que no es posible hacer suposiciones sobre las similitudes o diferencias entre los procesos en distintas compañías que acatan ISO 9001. Algunas compañías tienen procesos de calidad muy rígidos con registros detallados, mientras que otras son mucho menos formales, con poca documentación adicional.

En la figura siguiente se muestran las relaciones entre ISO 9001, manuales de calidad organizacional y planes de calidad de proyecto individuales. Este diagrama se derivó de un modelo presentado por Ince (1994), quien explica cómo puede usarse el estándar general ISO 9001 como base para procesos de gestión de calidad de software. Bamford y Dielbler (2003) explican cómo puede aplicarse el más reciente estándar ISO 9001:2000 en las compañías de software.




*ISO 9001 y gestión de la calidad*

Algunos clientes de software demandan que sus proveedores tengan la certificación ISO 9001. Así, los clientes podrán estar seguros de que la compañía que desarrolla el software **tiene un sistema de gestión de calidad aprobado**. Autoridades de acreditación independiente examinan los procesos de gestión de calidad y la documentación de pro- ceso, y deciden si dichos procesos abarcan todas las áreas especificadas en ISO 9001. Si es así, certifican que los procesos de calidad de una compañía, definidos en el manual de calidad, concuerdan con el estándar ISO 9001. Algunas personas consideran que la certificación ISO 9001 significa que la calidad del software producido por compañías certificadas será mejor que el derivado de compañías no certificadas.

Esto no precisamente es cierto. El estándar ISO 9001 se enfoca en garantizar que la organización tenga procedimientos de gestión de calidad y que siga dichos procedimientos. No hay seguridad de que las compañías con certificación ISO 9001 empleen las mejores prácticas de desarrollo de software o que sus procesos conduzcan a software de alta calidad.

Por ejemplo, una compañía podría definir estándares de cobertura de pruebas que especifiquen que todos los métodos en los objetos deben llamarse al menos una vez. Lamentablemente, este estándar puede cumplirse mediante pruebas de software incompletas, que no incluyen pruebas con diferentes parámetros de métodos. En tanto se sigan los procedimientos de prueba definidos y se conserven registros de las pruebas realiza- das, la compañía podría tener la certificación ISO 9001.

Esta certificación define la calidad como la conformidad con estándares, y toma en cuenta la calidad como la advierten los usuarios del software. Los **métodos ágiles, que evitan la documentación** y **se enfocan en el código a desarrollar**, tienen poco en común con los procesos de calidad formal que se examinan en ISO 9001. Se ha hecho cierto trabajo para reconciliar estos enfoques (Stalhane y Hanssen, 2008), pero la comunidad de desarrollo ágil en general se opone a lo que considera una carga burocrática de la conformidad con los estándares. Por esta razón, l**as compañías que usan métodos de desarrollo ágil se preocupan pocas veces por la certificación ISO 9001.**

## b. NMX-I-9126-2-NYCE-2011 | Certificación de Producto de Software

**Certificación de Producto de Software**

Es el proceso mediante el cual NYCE, da constancia de que un producto de software cumple con las especificaciones establecidas en la norma Mexicana NMX-I-9126-2-NYCE-2011, esto con base en la información técnica de dichos productos y en los resultados de los informes de pruebas realizados por un laboratorio reconocido o acreditado.

**Objetivo de la certificación**

La certificación de productos de software busca asegurar que se hayan hecho las pruebas de Funcionalidad mencionadas en la norma Mexicana que involucra características como: adecuación, exactitud, interoperabilidad, seguridad de acceso y cumplimiento funcional. Todo ello con la finalidad de que el software se considere de calidad.

**¿Por qué es importante evaluar su Producto de Software?**

- Diferenciarse de los competidores, asegurando tiempos de entrega y reducción de fallos en el producto tras su implantación en producción.
- Establecer acuerdos en el ámbito del servicio, definiéndose determinados parámetros de calidad que el producto debe cumplir antes de ser entregado.
- Detectar los defectos en el producto software y proceder a su eliminación antes de la entrega, lo que supone un ahorro de costes en la fase de mantenimiento posterior.
- Evaluar y controlar el rendimiento del producto software desarrollado, asegurando que podrá generar los resultados teniendo en cuenta las restricciones de tiempo y recursos establecidas.
- Asegurar que el producto software desarrollado respeta los niveles necesarios para las características de seguridad (confidencialidad, integridad, autenticidad, no-repudio, etc.)
- Comprobar que el producto desarrollado podrá ser puesto en producción sin poner en compromiso el resto de sistemas y manteniendo la compatibilidad con las interfaces necesarias.

| *Beneficios* | |
|---|---|
| **Para las empresas que desarrollan software** | **Para las empresas que adquieren software:** |
| • Controlar la calidad del producto desarrollado.<br>• Mejorar las características del producto.<br>• Asegurar a sus clientes un nivel de calidad.<br>• Comparar sus productos, con los productos de la competencia.<br>• Posicionar su producto en el mercado.<br>• Aumentar las ventas del producto. | • Conocer la calidad del producto que compran.<br>• Comparar entre distintas alternativas.<br>• Establecer acuerdos de nivel de servicio.<br>• Conseguir independencia del proveedor.<br>• Minimizar los fallos en producción.<br>• Reducir los costos finales al momento de comprar software. |

## c. NMX-I-059/02-NYCE-2016 – MoProSoft

En **NYCE (Normalización y Certificación, México)** elaboran y evalúan el estándar o norma **NMX-I-059/02-NYCE-2016** ***"Tecnología de la información – Ingeniería de Software – Calidad de producto" (MoProSoft)***, lo que permite brindar la legitimidad y certeza jurídica al modelo de evaluación de madurez de la capacidad de procesos para poder elevarlo a la categoría de norma, hoy estándar **MoProSoft.**

**MoProSoft** permite a las pequeñas y medianas empresas que desarrollan software, demostrar la capacidad de sus procesos y, con esto, hacerlas más competitivas, a fin de que tengan mayores probabilidades de permanecer en el mercado. Tiene un bajo costo, tanto para su capacitación, adopción como para su evaluación MoProSoft es un modelo de referencia de procesos conformado por un conjunto de buenas prácticas y métodos de gestión e ingeniería de software, que contribuyen a que las organizaciones dedicadas al desarrollo y mantenimiento de estas aplicaciones, mejoren su forma de trabajar y gestionar sus proyectos y por consiguiente incrementar sus niveles de capacidad y competitividad tanto nacional como internacionalmente.El esquema MoProSoft permite a las pequeñas y medianas empresas que desarrollan software, demostrar la capacidad de sus procesos y, con esto, hacerlas más competitivas, a fin de que tengan mayores probabilidades de permanecer en el mercado. Está enfocado hacia una de las estrategias del **Programa de Software (ProSoft)** de la Secretaría de Economía, relativas a "alcanzar niveles internacionales de capacidad de procesos" por parte de las pequeñas y medianas empresas mexicanas desarrolladoras.

**Características**

- Es específico para el desarrollo y mantenimiento de software.
- Facilita el cumplimiento de los requisitos de otros modelos como ISO/IEC 29110, ISO 9001:2008, ISO/IEC 15504, CMMI-DEV e ISO/IEC 12007.
- Es sencillo de entender y adoptar.
- Es práctico en su aplicación.
- Comprende un documento de menos de 200 páginas que al compararlo con otros modelos y estándares, lo hace bastante práctico.
- Resulta acorde con la estructura de las organizaciones mexicanas con desarrollo o mantenimiento de software.
- Está orientado a mejorar los procesos, para contribuir a los objetivos de la organización, y no simplemente ser un marco de referencia o dictaminación.

**Servicios adicionales sobre dicha Norma:**

- El modelo MoProSoft está dirigido a las empresas o áreas internas dedicadas al desarrollo y/o mantenimiento de software.
- Agrupa los procesos en tres categorías principales: Alta Dirección, Gerencia y Operación, entre otros.
- Las organizaciones que no cuentan con procesos establecidos pueden usar el modelo como la primera versión de sus procesos e ir ajustándolos de acuerdo a sus necesidades y experiencia adquirida.
- Las organizaciones que ya tienen procesos establecidos pueden usarlo como punto de referencia para identificar los elementos que les hace falta cubrir.

**Entregables**

- Informe de Verificación de Procesos de Tecnologías de la Información.
- Dictamen de Conformidad Legal (Certificado).
- Reconocimiento del Dictamen de Conformidad.
- Registro de la Empresa en el Padrón Nacional de Empresas de Dictaminadas.

**Registro ante la Entidad Mexicana de Acreditación (EMA)**

De acuerdo a los registros de la Entidad Mexicana de Acreditación (EMA) al día de hoy, Normalización y Certificación NYCE, S.C. (NYCE) es la única Unidad de Verificación de T.I., con **Número de Acreditación: UVTI 001**, que se encuentra legalmente acreditada para ofrecer los servicios de forma Nacional e Internacional sobre las Norma NMX-I-059/02-NYCE-2011 (MoProSoft).




## 5. Medición y métricas del software

La **medición del software** se ocupa de derivar un **valor numérico o perfil para un atributo de un componente, sistema o proceso de software**. Al comparar dichos valores unos con otros, y con los estándares que se aplican a través de una organización, es posible extraer conclusiones sobre la calidad del software, o valorar la efectividad de los procesos, las herramientas y los métodos de software.

Por ejemplo, suponga que una organización pretende introducir una nueva herramienta de prueba de software. Antes de introducir la herramienta, hay que registrar el número de defectos descubiertos de software en un tiempo determinado. Ésta es una línea de referencia para valorar la efectividad de la herramienta. Después de usar la herramienta durante algún tiempo, se repite este proceso. Si se descubren más defectos en el mismo lapso, después de introducida la herramienta, usted tal vez determine que ofrece apoyo útil para el proceso de validación del software.

La meta a largo plazo de la medición del software es usar la medición en lugar de revisiones para realizar juicios de la calidad del software. Al usar medición de software, un sistema podría valorarse preferentemente mediante un rango de métricas y, a partir de dichas mediciones, se podría inferir un valor de calidad del sistema. Si el software alcanzó un umbral de calidad requerido, entonces podría aprobarse sin revisión. Cuando es adecuado, las herramientas de medición pueden destacar también áreas del software susceptibles de mejora. Sin embargo, aún se está lejos de esta situación ideal y no hay señales de que la valoración automatizada de calidad será en el futuro una realidad previsible.

Una **métrica de software** es una característica de un sistema de software, documentación de sistema o proceso de desarrollo que puede medirse de manera objetiva. Los ejemplos de métricas incluyen el tamaño de un producto en líneas de código; el índice Fog (Gunning, 1962), que es una medida de la legibilidad de un pasaje de texto escrito; el número de fallas reportadas en un producto de software entregado, y el número de días- hombre requerido para desarrollar un componente de sistema.

Las **métricas de software** pueden ser métricas de **control o de predicción**.

Las métricas de **control** apoyan la **gestión del proceso**, y las **métricas de predicción** ayudan a predecir las **características del software.** Las **métricas de control** se asocian por lo general con **procesos de software**.

- Ejemplos de las métricas de control o de proceso son el esfuerzo promedio y el tiempo requerido para reparar los defectos reportados. Las métricas de predicción se asocian con el software en sí y a veces se conocen como métricas de producto.

- Ejemplos de métricas de predicción son la complejidad ciclomática de un módulo, la longitud promedio de los identificadores de un programa, y el número de atributos y operaciones asociados con las clases de objetos en un diseño.

Tanto las métricas de control como las de predicción pueden influir en la toma de decisiones administrativas. Los administradores usan mediciones de proceso para decidir si deben hacerse cambios al proceso, y las métricas de predicción ayudan a estimar el esfuerzo requerido para hacer cambios al software.

Existen **dos formas en que pueden usarse las mediciones de un sistema de software**:

1. Para asignar un valor a los atributos de calidad del sistema. Al medir las características de los componentes del sistema, como su complejidad ciclomática, y luego agregar dichas mediciones, es posible valorar los atributos de calidad del sistema, tales como la mantenibilidad.
2. Para identificar los componentes del sistema cuya calidad está por debajo de un estándar. Las mediciones pueden identificar componentes individuales con características que se desvían de la norma. Por ejemplo, es posible medir componentes para descubrir aquéllos con la complejidad más alta. Éstos tienen más probabilidad de tener bugs porque la complejidad los hace más difíciles de entender.

Es **difícil hacer mediciones directas de muchos de los atributos de calidad del software**. Los atributos de calidad, como mantenibilidad, comprensibilidad y usabilidad, son atributos externos que se refieren a cómo los desarrolladores y usuarios experimentan el software. Se ven afectados por factores subjetivos, como la experiencia y educación del usuario, y, por lo tanto, no pueden medirse de manera objetiva.

Para hacer un juicio sobre estos atributos, hay que medir algunos atributos internos del software (como tamaño, complejidad, etc.) y suponer que éstos se relacionan con las características de calidad por las que uno se interesa.

La figura siguiente muestra algunos atributos externos de calidad del software y atributos internos que podrían, intuitivamente, relacionarse con ellos. Aunque el diagrama sugiere que pueden existir relaciones entre atributos externos e internos, no dice cómo se relacionan dichos atributos. Si la medida del atributo interno debe ser un factor de predicción útil de la característica externa del software, deben sostenerse tres condiciones (Kitchenham, 1990):

1. El atributo interno debe medirse con exactitud. Esto no siempre es un proceso directo y tal vez se requiera de herramientas de propósito especial para hacer las mediciones.
2. Debe existir una relación entre el atributo que pueda medirse y el atributo de calidad externo que es de interés. Esto es, el valor del atributo de calidad debe relacionarse, en alguna forma, con el valor del atributo que puede medirse.
3. Esta relación entre los atributos interno y externo debe comprenderse, validarse y expresarse en términos de una fórmula o un modelo. La formulación de un modelo implica identificar la manera funcional del modelo (lineal, exponencial, etc.) mediante el análisis de datos recopilados, identificar los parámetros que se incluirán en el modelo, y calibrar dichos parámetros usando los datos existentes.




***Mediciones de predicción y de control***

Los atributos de software internos, como la complejidad ciclomática de un componente, se miden usando herramientas de software que analizan el código fuente del software. Hay herramientas disponibles de fuente abierta que pueden utilizarse para hacer dichas mediciones. Podría existir una relación entre la complejidad de un componente de software y el número de fallas observadas en el uso, es difícil demostrar objetivamente que éste es el caso. Para probar esta hipótesis, se requieren datos de falla para un gran número de componentes y acceso al código fuente del componente para su análisis. Muy pocas compañías han establecido un compromiso a largo plazo para la recopilación de datos sobre su software, de manera que pocas veces están disponibles datos de fallas para el análisis.

En la década de 1990, numerosas y grandes compañías, como Hewlett-Packard (Grady, 1993), AT&T (Barnard y Price, 1994) y Nokia (Kilpi, 2001) introdujeron programas de métricas. Hicieron mediciones de sus productos y procesos y las usaron durante sus procesos de gestión de calidad. La mayor parte de la atención se centró en la recolección de métricas sobre los defectos de programa y los procesos de verificación y validación. Offen y Jeffrey (1997) y Hall y Fenton (1997) tratan con más detalle la introducción en la industria de programas de métricas.

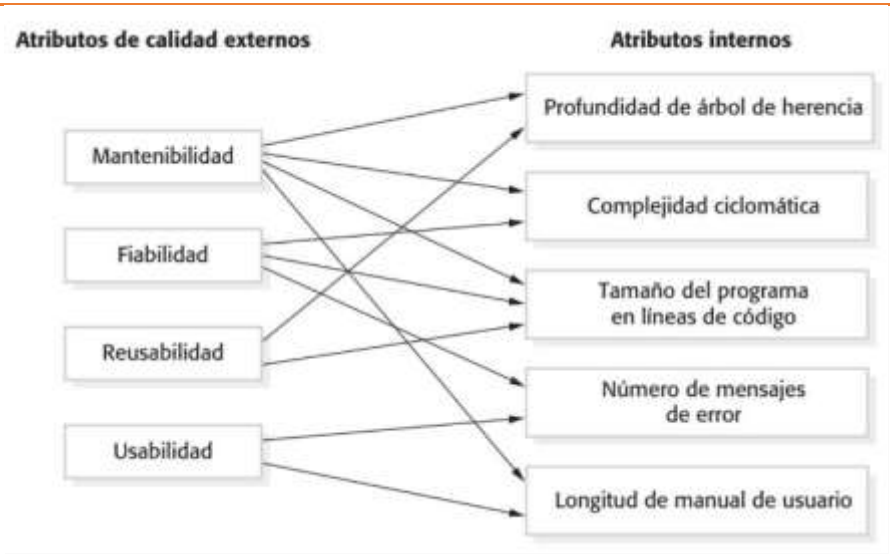
Existe **escasa información disponible al público concerniente al uso actual en la industria de la medición sistemática del software**. Muchas compañías reúnen información referente a su software, como el número de peticiones de cambio de requerimientos o el número de defectos descubiertos en las pruebas. Sin embargo, no es claro si usan entonces dichas mediciones de manera sistemática para comparar productos y procesos de software o para valorar el efecto de los cambios sobre los procesos y las herramientas de software. Existen algunas razones por las que esto se dificulta:

1. Es imposible cuantificar la rentabilidad de la inversión de introducir un programa de métricas organizacional. En años pasados existieron significativas mejoras en la calidad del software sin el uso de métricas, así que es difícil justificar los costos iniciales de introducir medición y valoración sistemáticas del software.
2. No hay estándares para las métricas de software o para los procesos estandarizados para medición y análisis. Muchas compañías son renuentes a introducir programas de medición hasta que se hallan disponibles tales estándares y herramientas de apoyo.
3. En gran parte de las compañías, los procesos de software no están estandarizados y se encuentran mal definidos y controlados. Por lo tanto, hay demasiada variabilidad de procesos dentro de la misma compañía para que las mediciones se usen en una forma significativa.
4. Buena parte de la investigación en la medición y métricas del software se enfoca en métricas basadas en códigos y procesos de desarrollo basados en un plan. Sin embargo, ahora cada vez más se desarrolla software mediante la configuración de sistemas ERP o COTS, o el uso de métodos ágiles.
5. Por consiguiente, no se sabe si la investigación previa es aplicable a dichas técnicas de desarrollo de software.

La introducción de medición representa una carga adicional a los procesos. Esto contradice las metas de los métodos ágiles, los cuales recomiendan la eliminación de actividades de proceso que no están directamente relacionadas con el desarrollo de pro- gramas. En consecuencia, es improbable que las compañías que adoptaron los métodos ágiles aprueben un programa de métricas.

La medición y las métricas de software son la base de la ingeniería de software empírica (Endres y Rombach, 2003). Ésta es un área de investigación en la que se han usado experimentos respecto a los sistemas de software, y la recolección de datos referente a proyectos reales para formar y validar hipótesis sobre métodos y técnicas de ingeniería de software. Los investigadores que trabajan en esta área argumentan que sólo es posible confiar en el valor de los métodos y las técnicas de la ingeniería de software si se encuentra evidencia concreta de que en realidad ofrecen los beneficios que sugieren sus inventores.

Es lamentable que aun cuando es posible hacer mediciones objetivas y extraer conclusiones a partir de ellas, esto no necesariamente convence a quienes toman las decisiones. En vez de ello, la toma de decisiones está influida con frecuencia por factores subjetivos, como la novedad, o la medida en que las técnicas son de interés para los profesionales. Por lo tanto, se considera que transcurrirán muchos años antes de que los resultados de la ingeniería de software empírica presenten un efecto significativo sobre la práctica de la ingeniería de software.




***Relaciones entre software interno y externo***

**Métricas del producto**

Las **métricas del producto** se emplean para medir los atributos internos de un sistema de software. Los ejemplos de las métricas de productos incluyen el tamaño del sistema, la medida en líneas de código o el número de métodos asociados con cada clase de objeto. Las características del software que pueden medirse fácilmente, como el tamaño y la complejidad ciclomática, no tienen una relación clara y consistente con los atributos de calidad como comprensibilidad y mantenibilidad. Las relaciones varían dependiendo de los procesos de desarrollo, la tecnología empleada y el tipo de sistema a diseñar.

Las métricas del producto se dividen en dos clases:

1. **Métricas dinámicas**, que se recopilan mediante mediciones hechas de un programa en ejecución. Dichas métricas pueden recopilarse durante las pruebas del sistema o después de que el sistema está en uso. Un ejemplo es el número de reportes de bugs o el tiempo necesario para completar un cálculo.
2. **Métricas estáticas**, las cuales se recopilan mediante mediciones hechas de representaciones del sistema, como el diseño, el programa o la documentación. Ejemplos de mediciones estáticas son el tamaño del código y la longitud promedio de los identificadores que se usaron.

Estos tipos de métrica se relacionan con diferentes atributos de calidad. Las métricas dinámicas ayudan a valorar la eficiencia y fiabilidad de un programa. Las métricas estáticas ayudan a valorar la complejidad, comprensibilidad y mantenibilidad de un sistema de software o de los componentes del sistema. Por lo general, existe una relación clara entre métricas dinámicas y características de calidad del software. Es muy sencillo medir el tiempo de ejecución requerido para funciones particulares y valorar el tiempo requerido con la finalidad de iniciar un sistema. Éstos se relacionan directamente con la eficiencia del sistema.

Las métricas estáticas, tienen una relación indirecta con los atributos de calidad. Se ha propuesto una gran cantidad de diferentes métricas y se han intentado muchos experimentos para derivar y validar las relaciones entre dichas métricas y atributos como complejidad y mantenibilidad. Ninguno de tales experimentos ha sido concluyente, pero el tamaño del programa y la complejidad del control parecen ser los factores de predicción más fiables de la comprensibilidad, la complejidad del sistema y la mantenibilidad.

Las métricas de la figura siguiente son aplicables a cualquier programa, pero también se han propuesto métricas más específicas orientadas a objetos (**OO**).

En esta tabla se resume la suite de Chidamber y Kemerer (en ocasiones llamada suite **CK**) de seis métricas orientadas a objetos (1994). Aunque se propusieron originalmente a principio de la década de 1990, aún son las métricas OO de más amplio uso. Algunas herramientas de diseño UML recopilan automáticamente valores para dichas métricas conforme se crean los diagramas UML.

El-Amam (2001) hace una excelente revisión de las **métricas orientadas a objetos**, analiza las métricas **CK** y otras métricas OO, y concluye que todavía no se tiene suficiente evidencia para comprender cómo estas y otras métricas orientadas a objetos se relacionan con cualidades externas de software. Esta situación no ha cambiado realmente desde su análisis en 2001. Todavía no se sabe cómo usar las mediciones de los programas orientados a objetos para extraer conclusiones fiables acerca de su calidad.

| Métrica de software | Descripción |
|---|---|
| Fan-in/Fan-out | Fan-in (abanico de entrada) es una medida del número de funciones o métodos que llaman a otra función o método (por ejemplo, X). Fan-out (abanico de salida) es el número de funciones a las que llama la función X. Un valor alto para fan-in significa que X está estrechamente acoplado con el resto del diseño y que los cambios a X tendrán extensos efectos dominó. Un valor alto de fan-out sugiere que la complejidad global de X puede ser alta debido a la complejidad de la lógica de control necesaria para coordinar los componentes llamados. |
| Longitud de código | Ésta es una medida del tamaño de un programa. Por lo general, cuanto más grande sea el tamaño del código de un componente, más probable será que el componente sea complejo y proclive a errores. Se ha demostrado que la longitud del código es una de las métricas más fiables para predecir la proclividad al error en los componentes. |
| Complejidad ciclomática | Ésta es una medida de la complejidad del control de un programa. Tal complejidad del control puede relacionarse con la comprensibilidad del programa. En el capítulo 8 se estudia la complejidad ciclomática. |
| Longitud de identificadores | Ésta es una medida de la longitud promedio de los identificadores (nombres para variables, clases, métodos, etcétera) en un programa. Cuanto más largos sean los identificadores, es más probable que sean significativos y, por ende, más comprensible será el programa. |
| Profundidad de anidado condicional | Ésta es una medida de la profundidad de anidado de los enunciados if en un programa. Los enunciados if profundamente anidados son difíciles de entender y proclives potencialmente a errores. |
| Índice Fog | Ésta es una medida de la longitud promedio de las palabras y oraciones en los documentos. Cuanto más alto sea el valor del índice Fog de un documento, más difícil será entender el documento. |

***Métricas estáticas de productos de software***

**Nombre(s):** _____ **No. Lista** _____ **Fecha:** _____ **Grupo:** _____
**(Apellido paterno. Apellido materno. Nombre(s)**

## RESPONDE

**1.** ¿En qué consiste la **gestión de calidad** en el software?

**2.** Explique cuál es la función de **NYCE.**

**3.** Explique en qué consiste la Norma: **NMX-I-9126-2-NYCE-2011.**

**4.** ¿Cuál de los términos siguientes es el indicado para referirse a la **calidad** en la industria del software?

☐ Control de calidad ☐ Aseguramiento de calidad

**5.** Mencione cuáles son las 4 **características esenciales de un sistema de software profesional**.

## RESPONDE

**6.** ¿Qué **preguntas formularía** para justificar el argumento siguiente?

*"La valoración de calidad del software es un proceso subjetivo en que el equipo de gestión de calidad tiene que usar su juicio para decidir si se logró un nivel aceptable de calidad. El equipo de gestión de calidad debe considerar si el software se ajusta o no a su propósito pretendido. Esto implica responder preguntas sobre las características del sistema".*

**7.** Mencione **5 atributos de calidad del software.**

**8.** ¿Por qué se afirma que "**el software no se manufactura, se diseña"**?

**9.** ¿Qué son los **estándares de software** y porqué son importantes?

**10.** ¿Qué es la **ISO 9001** y para qué sirve?

## RESPONDE

**11.** ¿Cuál sería la justificación de que dependencias o compañías como el **Instituto Electoral del Distrito Federal** (ahora CDMX), **Insignia IT, S. A. de C. V., LYF Ingeniería, S.A. de C.V**. se encuentren en el **Padrón de compañías o dependencias dictaminadas en la norma NMX-I-059/02-NYCE (MoProSoft)?**

**12.** ¿Por qué se afirma que es **difícil hacer mediciones directas de muchos de los atributos de calidad del software**?

**13.** Mencione cuáles son los **dos tipos de estándares** de ingeniería de software.

**14.** ¿Cuál es la función de las **métricas del producto?**

**15.** ¿Cuál es la **norma** que permite especificar y evaluar la calidad del software desde diferentes criterios asociados con adquisición, requerimientos, desarrollo, uso, evaluación, soporte, mantenimiento, aseguramiento de la calidad y auditoria de software?

## RESPONDE

**16.** Presente la siguiente **Solicitud General para la certificación de producto de software** como un ejercicio asumiendo la función de "Integrador de sistemas" y requiere la certificación de un producto de software.

ATENCIÓN

**IMPORTANTE**: No proporcione para este ejercicio sus datos personales.

- **a.** Nombre/Razón social de la empresa: (Letras de sus apellidos + nombre, por ejemplo: **Pe**dro **Pé**rez **Sa**ndoval = **PEPESA, TI, SA de CV.**
- **b.** Domicilio: Av. Té 950…Iztacalco (Dirección y datos de UPIICSA)

CLICK

Organismo de normalización y de evaluación de la conformidad
e-mail: nyce@nyce.org.mx www.nyce.org.mx

NYCE
NORMALIZACIÓN Y CERTIFICACIÓN NYCE, S.C.

### SOLICITUD GENERAL PARA LA CERTIFICACIÓN DE PRODUCTO DE SOFTWARE

FR1CCS1

| **No. de Solicitud:** | **Fecha:** | **Revisó:** | **Supervisó:** |
|---|---|---|---|

(Datos a ser llenados por NYCE)

**TIPO DE SERVICIO**

☐ NMX-I-9126-2-NYCE-2011 – Tecnología de la Información – Ingeniería de Software – Calidad de Producto –Parte 2: Métricas Externas.

☐ NMX-I-9126-2-NYCE-2011 – Tecnología de la Información – Ingeniería de Software – Calidad de Producto –Parte 2: Métricas Externas y los Parámetros de Autorregulación en Materia de Protección de Datos Personales.

**DATOS DEL CLIENTE**

Nombre y/o Razón Social de la empresa:

Calle: No. Ext.: No. Int.: Col. o Poblado:

C.P: Delegación o Municipio: Estado:

R.F.C.: Página Web: Correo electrónico:

| | |
|---|---|
| Representante legal: | Nombre del tramitador autorizado: |
| Tel.: ( ) (Con clave de larga distancia) | Tel.: ( ) (Con clave de larga distancia) |
| Correo electrónico: | Correo electrónico: |

## RESPONDE

| DATOS DEL PRODUCTO |
| --- |
| Nombre del producto de software: _____<br>Versión: _____ ID del fabricante (Cuando aplique): _____<br>Realice una breve descripción del producto que desea evaluar (propósito, características, etc.):<br>_____ |
| Indique lo requisitos necesarios para el uso del producto (sistema operativo, framework de ejecución necesario (p. ej.: JRE, .NET, etc.) y su versión, manejador de base de datos, etc.):<br>_____ |
| **DATOS ADICIONALES DEL PRODUCTO** |
| ¿En qué lenguaje(s) está desarrollado el producto de software que desea evaluar?<br><br>☐ Java<br>☐ .Net (C#, VB.NET)<br>☐ PHP<br>☐ C++<br>☐ Objetive-C<br>☐ Groovy<br>☐ JavaScript<br>☐ Android<br>☐ Otros (especifique cuáles): _____ |
| Indique por qué medio entregara el producto de software a evaluar (CD, DVD, USB, etc): _____ |

**RESPONDE**

Indique que información del producto de software puede aportar para la evaluación:

- ☐ Especificación de Requisitos (ERS)*
- ☐ Análisis y Diseño (Arquitectónico y Detallado)*
- ☐ Diccionario de Datos y Base de Datos
- ☐ Descripción funcional con pantallas
- ☐ Documentación técnica (diagramas UML, Casos de Uso, etc.)*
- ☐ Manual de Usuario*
- ☐ Manual de Operación
- ☐ Conjunto de Prueba Unitarias creadas para la Aplicación

*Nota: Para la realización de la evaluación es imprescindible contar está información.

Nota: La documentación técnica no será evaluado. Únicamente se utilizara para entender mejor el producto de software a evaluar, su alcance y así poder realizar una cotización formal.

¿Cuál es el tamaño del producto de software a evaluar? Indique los valores de los que disponga

Número de Requisitos Funcionales: _____
Número de Líneas de Código: _____
Número de Casos de Uso: _____
Número de Casos de Prueba.: _____

Indique el número de instalaciones del producto: _____

Indique el tiempo del producto de software en el mercado: _____

**DOCUMENTACIÓN LEGAL REQUERIDA**

| | |
|---|---|
| Acta constitutiva de la empresa. | |
| Registro Federal de Contribuyentes. | |
| Alta ante Hacienda. | |
| Comprobante de Domicilio. | |
| Identificación Oficial con fotografía del Representante legal | |
| Otro (indique). | |

**Nota:** En caso de que su organización ya sea usuario de NYCE y haya entregado la documentación arriba mencionada, favor de hacer caso omiso a este apartado a menos que haya sufrido algún cambio en la información previamente entregada

**COMENTARIOS ADICIONALES:**

_____

Representante Autorizado de la Organización

_____

(Nombre y firma)